

TOOL SELECTION

GitHub Copilot と Claude Code を どう使い分けるか

できることの重なりと違い、課金モデルの前提、社内での配置方針



比較の前提

両者はいまや同じ領域で競合している。「補完 対 エージェント」という古い対比は成立しない

01 どちらも自律実行するエージェントを持つ

計画・編集・テスト・PR 作成まで一連で行える点は共通している

02 違いは能力の有無ではなく、置かれている場所

Copilot はリポジトリの中、Claude Code は端末と作業環境の側に軸足がある

03 併用は前提として考える

排他的に 1 つへ寄せる判断は、どちらの強みも捨てることになる

作業の性質による使い分け

自律度：まとめて任せる

Copilot

Issue から PR まで任せる

課題を割り当てて背後で実装させる。GitHub の中で完結する

Claude Code

調査・設計から実装まで任せる

複数リポジトリ、CI、外部サービスを横断する作業

Copilot

補完とインラインの提案

手を動かしながら書く。クレジットを消費しない

Claude Code

対話しながら直す

端末で試しながら進める。ローカルの実行結果を見て判断する

自律度：人が逐次確認

作業範囲：リポジトリ内で閉じる

作業範囲：環境・ツールをまたぐ

定型で範囲が閉じた作業は Copilot、探索を伴い環境をまたぐ作業は Claude Code が向く

できることの対応関係

領域	GitHub Copilot	Claude Code
自律実行	コーディングエージェントが Issue から PR を作る。CLI には Plan ・ Autopilot モードがある	CLI が標準。計画モードと自動承認モードを持ち、作業環境の中で完結する
並列作業	CLI が Explore ・ Task ・ Code Review ・ Plan の専用サブエージェントを持つ	サブエージェントを自分で定義できる。ツール ・ モデル ・ 権限を個別に制限できる
組織の作法	AGENTS.md とカスタムエージェントでリポジトリ単位に定義する	Skills と CLAUDE.md で定義する。個人 ・ プロジェクト ・ 組織配布の 3 階層を持つ
コードレビュー	エージェント型レビューが指摘し、そのまま修正 PR を生成できる	レビュー機能は Team ・ Enterprise プラン。CI イベントに反応する運用も組める
外部連携	GitHub 公式 MCP サーバーを内蔵し、カスタム MCP も使える	MCP に対応。フックで任意のコマンドを工程へ差し込める
統制	コンテンツ除外 ・ 監査ログ ・ ポリシー管理を Business 以上で提供	managed settings で組織配布。サーバー管理設定はサブスクリプション契約が前提

機能の重なりは大きい。差が出るのは、組織の作法をどこまでツールへ持ち込めるかという点

それぞれが明確に強い領域

Copilot が強い領域

- **GitHub 上の作業との一体化**
Issue、PR、レビュー、Actions と同じ画面と権限の中で動く
- **補完の運用コストの低さ**
コード補完と Next Edit はクレジットを消費せず、全プランに含まれる
- **既存の GitHub 統制をそのまま使える**
監査ログとポリシーが GitHub Enterprise の仕組みに載る

Claude Code が強い領域

- **作業環境そのものを扱える**
端末、複数リポジトリ、外部ツールを横断して1つの作業として進められる
- **組織の作法を構造化して配布できる**
Skills は必要なときだけ読み込まれるため、量が増えても常時コストにならない
- **実行の制御を細かく設計できる**
サブエージェント単位でツール・モデル・権限・メモリを分けられる

GitHub の中で完結する運用は Copilot、環境と作法を自分たちで組み立てる運用は Claude Code に寄せる

課金の前提が 2026 年 6 月に変わった

1credit = \$0.01

Copilot は 2026 年 6 月 1 日に AI クレジットへ移行

従来のプレミアムリクエスト単位を置き換え、トークン消費に応じて課金される

1,900

Business （\$19/ 人月）に含まれるクレジット

3,900

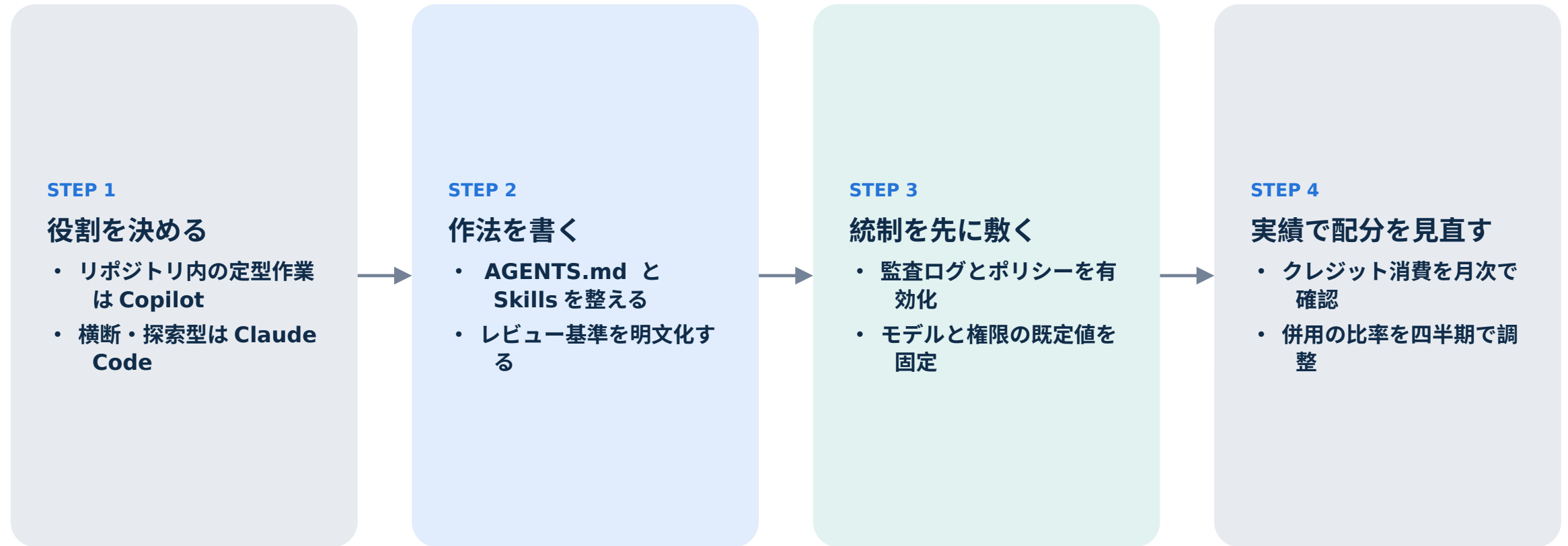
Enterprise （\$39/ 人月）に含まれるクレジット

0

コード補完・Next Edit の消費クレジット

Copilot は席料の中に従量枠を持つ形になった。単純な席数比較では費用を見誤る

社内での配置の進め方



両方を同時に全社展開せず、役割を決めてから広げる

推奨する方針

この判断の根拠

- **能力の重なりが大きく、優劣で決まらない**
自律実行・並列サブエージェント・MCP・カスタム作法は、いずれも両者が持っている
- **強みの源泉が構造的に違う**
Copilot は GitHub との一体化、Claude Code は環境の横断と作法の構造化にある。時間で埋まる差ではない
- **費用の形が異なる**
Copilot は席料に従量枠を内包し、補完は無償枠。使い方によって実効単価が大きく変わる
- **統制の置き場所が違う**
GitHub 側で統制するか、端末の設定配布で統制するかは、既存の運用体制で決まる

推奨

**併用を前提に、作業の性質で振り分ける。
どちらか一方への統一は費用も効果も悪化させる**

GitHub 上で完結する定型作業に Copilot、環境を横断する探索型の作業に Claude Code を割り当てる。まず 1 部門で 3 か月併用し、クレジット消費と作業時間の実績を見てから配分を決める。